SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and

XML, and apply Internet protocols and HTTP communication mechanisms for web content

delivery. (L2)

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: 5 **Total Marks: 20**

Code of Conduct:

I Challa Priyadarshini(192311118)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action**.**

Signature of the student: Challa Priyadarshini

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	5	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	4.5	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	4	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	3.5	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	2	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand
Total	19/20				

Question 1: HTTP Response Analysis

A web server returns the following HTTP response header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

1. HTTP Response Analysis

Given response:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1.1 Incorrect or invalid fields

- Content-Length: -150 is invalid. Content-Length represents the number of octets (bytes) in the message body and cannot be negative.
- Date: The weekday and calendar date do not match. 1 April 2026 is a Wednesday, not a Monday. A valid HTTP-date should use the correct weekday, for example: Wed, 01 Apr 2026 10:00:00 GMT.
- Content-Type: text/html is valid if the response body contains HTML. Adding a charset, such as text/html; charset=utf-8, is often helpful.
- Connection: keep-alive is used with HTTP/1.1 and is generally valid, though persistent connections are normally the default unless closed.
- HTTP/1.1 200 OK is syntactically valid, but its correctness depends on whether the server successfully fulfilled the request and whether the response metadata/body are consistent.

1.2 Content-Length issue and impact

A negative Content-Length is not a valid body length. It can cause a client, proxy, or validation tool to reject the response, report a protocol error, or behave inconsistently. The header must match the exact number of bytes in the response body—not the number of characters in every possible encoding.

1.3 Does status code 200 match?

200 OK means the request succeeded. The invalid Content-Length makes the response malformed, so the response as a whole is not correct even though 200 is a valid status code. A server should fix the response headers; changing the status code alone does not repair an invalid length.

1.4 Corrected example

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 46
Connection: keep-alive

<html><body><h1>Hello</h1></body></html>
```

Note: Content-Length: 46 is illustrative only. In a real response, calculate the exact byte length of the actual encoded body, or let the web server/framework generate it. Do not copy a sample length unless it has been verified.

1.5 Good practices

- Let a trusted HTTP server or framework calculate Content-Length automatically.
- Validate response headers and test responses with browser developer tools or an HTTP client.
- Ensure Date values use a valid HTTP-date format and the correct weekday.
- Avoid manually setting framing headers unless the application requires it.

Question 2: DOM Structure Analysis

Consider the following DOM tree representation:

<html>
<head>
<title>TestPage</title>
</head>
<body>
** **
<p>HelloWorld
** **
</body>
</html>

Questions:

1. Identify missing or improperly closed elements.

2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

2. DOM Structure Analysis

Given markup:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

2.1 Missing or improperly closed elements

- The paragraph element <p> is not explicitly closed before .
- The , <body>, and <html> elements have closing tags. The title and head are also closed correctly.

2.2 How the browser interprets it

HTML browsers use error-recovery rules and generally attempt to build a usable DOM rather than stopping at the first markup error. When the parser encounters while a paragraph is open, it will normally close the paragraph implicitly before closing the div. The exact DOM can be inspected in the browser's Elements panel.

2.3 Impact on rendering and layout

The page may still display “HelloWorld” as expected, so the error may not be visually obvious. However, relying on parser recovery makes the document harder to maintain and can lead to unexpected spacing, styling, accessibility, or scripting behavior as the markup grows.

2.4 Corrected HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TestPage</title>
</head>
<body>
  <div>
```

```
  <p>HelloWorld</p>
 </div>
</body>
</html>
```

The doctype selects standards mode; lang identifies the page language; charset defines text encoding; and the viewport meta tag supports responsive layouts.

Question 3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

3. GET vs POST in a Login System

Given examples:

Feature	**GET**	**POST**	**Security meaning**
Example	/login?user=admin&pass=1234	/login with form body	Credentials should not be in the URL
Where data goes	URL query string	HTTP request body	POST hides data from the address bar, not from all observers
Typical purpose	Retrieve/read resources	Submit data or trigger processing	Choose method based on semantics
Caching/history/logs	May be saved in history, logs, analytics and referrer data	Body is not normally in URL history, but may still be logged	Both need secure handling

Encryption	None by itself	None by itself	HTTPS/TLS is essential for both

3.1 Transmission differences

GET places submitted parameters in the URL query string. This makes them visible in the address bar and potentially in browser history, server logs, proxy logs, analytics, and referrer information. POST normally places submitted form data in the request body. That avoids exposing credentials in the URL, but does not encrypt the body.

3.2 Most suitable method for login

POST is generally appropriate for submitting login credentials because credentials should not be part of the URL. However, POST alone is not a security mechanism. Use HTTPS, secure session handling, and server-side authentication controls.

3.3 Risks of GET for authentication

- Credentials may remain in browser history, bookmarks, screenshots, copied links, and monitoring logs.
- URLs may be shared accidentally or captured by analytics and referrer handling.
- Credentials can be exposed to anyone with access to relevant logs or browser history.
- Without HTTPS, network observers may read both URL and body traffic.

3.4 Secure login example

```
<!-- HTML form -->
<form action="/login" method="post">
  <label for="username">Username</label>
  <input id="username" name="username" autocomplete="username" required>

  <label for="password">Password</label>
  <input id="password" name="password" type="password"
        autocomplete="current-password" required>

  <button type="submit">Sign in</button>
</form>
```

3.5 Best practices

- Serve the login page and submit credentials only over HTTPS.
- Store passwords using a strong password-hashing algorithm such as Argon2id, bcrypt, or scrypt with appropriate parameters and unique salts; never store plaintext passwords.
- Use secure, HttpOnly, and appropriately configured SameSite session cookies; rotate session identifiers after login.
- Apply rate limiting, generic error messages, multi-factor authentication where appropriate, and monitoring for suspicious attempts.
- Protect state-changing actions against CSRF where relevant; validate inputs on the server.

- Never put passwords, session tokens, or API secrets in URLs. Redact sensitive fields from logs.

Question 4: HTML Code Inspection

Analyze the following HTML snippet:

<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
** This is a paragraph**
<img src="image.jpg">
</body>
</html>

Questions:

1. Identify missing closing tags.
2. Analyze issues with the <img> tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

4. HTML Code Inspection

Given snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

4.1 Missing closing tags

- The <h1> heading is missing an explicit </h1>.
- The paragraph is missing an explicit .
- The <img> element is a void element in HTML and does not need a closing tag. <img /> is also accepted in HTML, but the slash is unnecessary.

4.2 Issues with the image element

- The image has no alt attribute. Meaningful images should have descriptive alternative text; decorative images should use alt="".
- If image.jpg is missing or the path is wrong, the browser cannot display the image.
- Width and height attributes can reserve layout space and reduce layout shifts. Use dimensions that match the image's aspect ratio.

4.3 Browser behavior

Browsers usually recover from omitted heading and paragraph end tags and still render the content. Nevertheless, the resulting DOM is produced using HTML parsing rules, and the source becomes less clear to people and tools. An image without alt text is also less accessible to users of screen readers.

4.4 Corrected HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
  <img src="image.jpg" alt="Description of the image" width="600" height="400">
</body>
</html>
```

Replace the sample alt text and dimensions with values that accurately describe the real image. If the image is purely decorative, use alt="" instead.

4.5 Validation checklist

- Use a doctype, a language attribute, a character encoding declaration, and a descriptive title.
- Close non-void elements explicitly and nest elements correctly.
- Give informative images suitable alternative text.
- Check the page using an HTML validator and browser developer tools.

Question 5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser

2. DNS lookup occurs

3. HTTP request is sent

4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

5. Browser–Server Communication Flow

5.1 Explanation of each step

1. User enters a URL: The browser parses the scheme (HTTP/HTTPS), hostname, port if supplied, path, and query. It checks whether a usable cached response or connection can be reused.

2. DNS lookup occurs: The browser/operating system resolves the hostname to an IP address, often using caches before contacting a DNS resolver.

3. HTTP request is sent: The browser establishes or reuses a connection. HTTPS adds a TLS handshake when needed. It then sends a request containing a method, path, headers, and optionally a body.

4. Server processes request: The web server or application routes the request, checks permissions, may query a database or another service, and creates a response.

5. Response is returned: The server sends a status code, headers, and optional body. The response may include HTML, JSON, images, CSS, JavaScript, or a redirect.

6. Browser renders page: The browser parses HTML into the DOM, parses CSS into the CSSOM, runs applicable JavaScript, lays out elements, paints pixels, and requests additional resources as needed.

5.2 Where delays can occur

- DNS: cache miss, slow resolver, or DNS network delay.
- Connection setup: TCP handshake and, for HTTPS, TLS negotiation; additional cost can occur when a connection cannot be reused.
- Network transfer: high latency, packet loss, low bandwidth, or large response payloads.
- Server processing: slow application code, overloaded server, database queries, or third-party API calls.
- Rendering: large CSS/JavaScript files, render-blocking resources, expensive JavaScript, large images, and layout recalculation.

5.3 Role of DNS

DNS (Domain Name System) maps a human-readable hostname such as www.example.com to network information, commonly an IP address. It allows users to use memorable domain names rather than numerical addresses. DNS resolution does not itself download the web page; it helps the browser locate the destination server.

5.4 Performance optimizations

- Use DNS caching and a reliable DNS provider; configure DNS records appropriately.
- Use HTTPS with modern protocol support and connection reuse; HTTP/2 or HTTP/3 may improve multiplexing and transport performance when supported.
- Compress text responses with Brotli or gzip and reduce unnecessary payloads.
- Optimize and appropriately size images; use modern formats such as WebP or AVIF where suitable, and lazy-load below-the-fold images.
- Minify and split CSS/JavaScript sensibly; defer non-critical scripts and reduce render-blocking resources.
- Use browser caching with correct Cache-Control headers for static assets; use a CDN for geographically distributed users.
- Optimize server code and database queries; add indexes where appropriate and cache expensive repeated results.
- Measure performance using browser DevTools Network/Performance panels and metrics such as LCP, INP, and CLS.

5.5 Simplified flow diagram

```
User enters URL
    |
    v
Parse URL and check cache
    |
    v
Resolve hostname with DNS
    |
    v
Establish/reuse connection (TLS for HTTPS)
    |
    v
Send HTTP request  --->  Web server / application
                              |
                              v
                         Process request
                              |
                              v
Receive status + headers + body
    |
```

```
    v
Parse HTML/CSS/JS and fetch subresources
    |
    v
Layout, paint and display page
```

Conclusion

These five cases demonstrate that reliable web development requires correct HTTP metadata, well-structured HTML, safe handling of credentials, accessible markup, and an efficient browser–server workflow. Browsers often recover from mistakes, but developers should not depend on recovery behavior. Validate HTML, let server frameworks generate protocol metadata, use HTTPS and secure authentication practices, and measure performance before optimizing.

Suggested practical tools: browser Developer Tools (Elements, Network, Console, Performance), an HTML validator, and an HTTP client such as curl. Use only authorized test systems when inspecting network behavior.